

DAY 2 • LECTURE 1 • 70 MINUTES

The Radar Room

Kubernetes Architecture, Deployments & k9s

Yesterday you launched a Pod. Today you meet the harbour-master who runs the whole island.

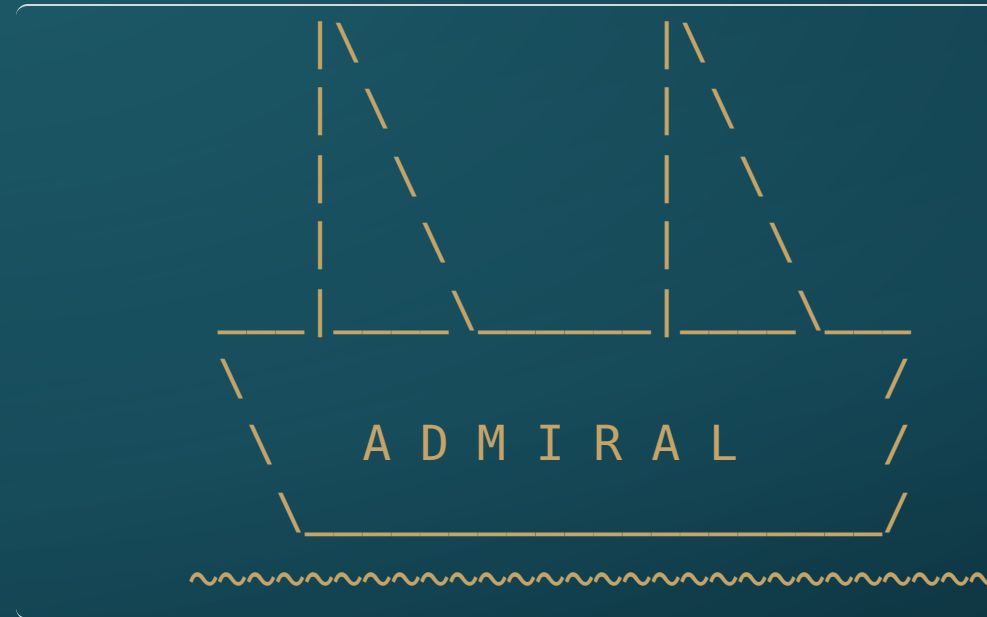


Where we are

- ♦ Yesterday you launched a **bare Pod** into the cluster — the smallest thing Kubernetes can run.
- ♦ The next 70 minutes: the full picture of what keeps that Pod alive, and the tools to see it.
- ♦ Four parts:
 - **Part I** — Meet Captain Kube (Kubernetes architecture)
 - **Part II** — From Pod to Deployment (resilience and self-healing)
 - **Part III** — Config lives off the ship (ConfigMaps)
 - **Part IV** — The Radar Room (k9s)
- ♦ Then the labs: **Lab 01** (find the broken ship) and **Lab 02** (build a resilient fleet).

PART I

Meet Captain Kube

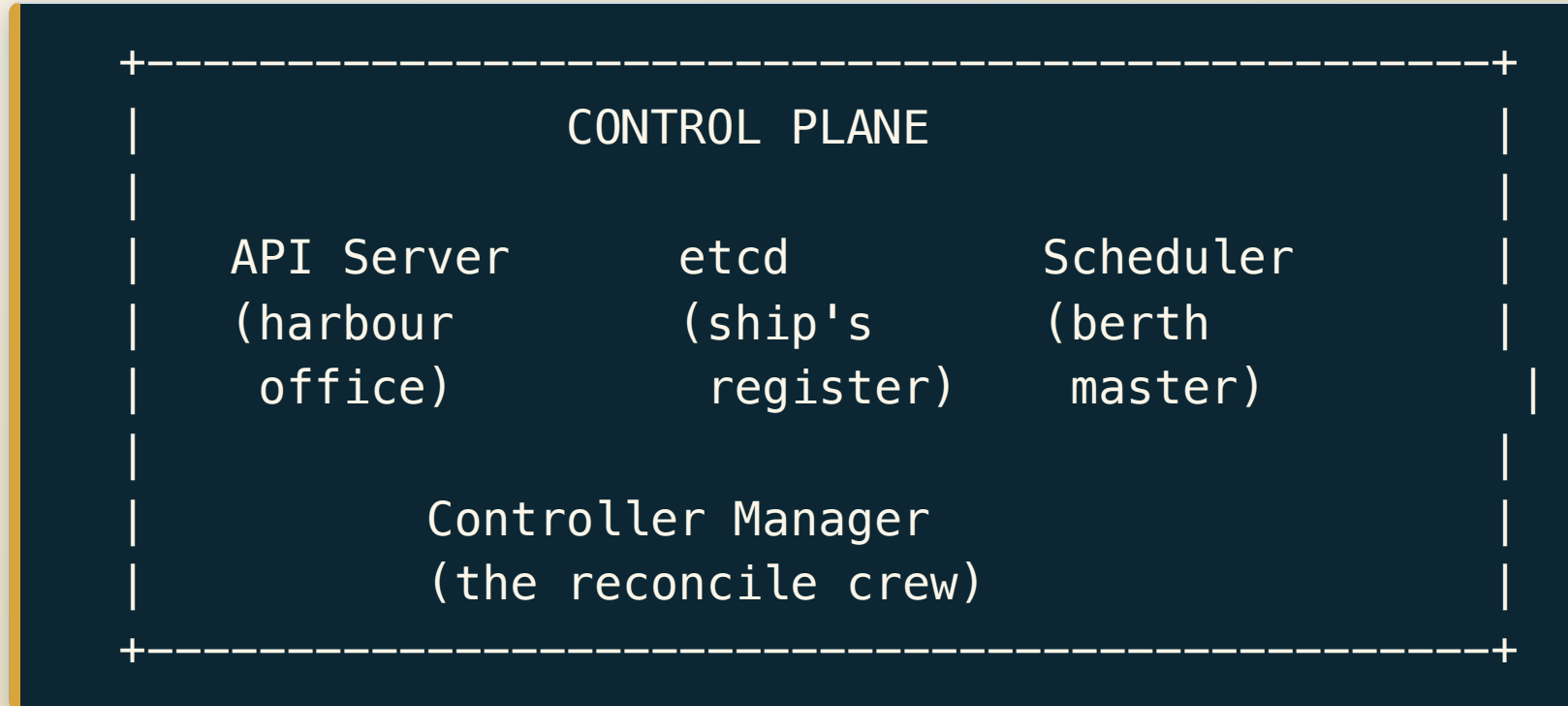


"The harbour-master sees every ship, every berth, every tide. That's not power — that's paperwork done right." — the Boatswain

The island's harbour-master

- ♦ On Day 1 you ran `kubectl apply` and a Pod appeared. Something made that happen.
- ♦ **Kubernetes** is the orchestrator running the island — the book calls it **Captain Kube**.
- ♦ Captain Kube has two halves:
 - the **Control Plane** — the harbour office that makes decisions
 - the **Worker Nodes** — the ships that carry the cargo
- ♦ Everything you do with `kubectl` is a message to the harbour office.

The Control Plane — the harbour office



- ♦ Four components, one purpose: keep the cluster matching what you asked for.

The API Server — harbour office counter

- ♦ **Every** request goes through the API Server — `kubectl`, the scheduler, the controllers, all of them.
- ♦ It is the single front door to the cluster.
- ♦ It validates your manifest, stores the result in etcd, and notifies the rest of the system.
- ♦ Think of it as the clerk at the harbour office: nothing moves without a stamp from this counter.

etcd — the ship's register

- ♦ The API Server writes every decision to **etcd** — a distributed key-value store.
- ♦ etcd is the cluster's **memory of desired state**: what you asked for, and what currently exists.
- ♦ If the whole control plane rebooted right now, etcd would bring it back to exactly where it was.
- ♦ One ledger. Every manifest ever applied lives here.

The crew can change; the ship's log endures.

The Scheduler — the berth master

- ♦ A new Pod lands in etcd: "I need a home." The **Scheduler** finds it one.
- ♦ It looks at every worker node: How much CPU and memory is free? Any constraints on this Pod?
- ♦ It picks the best berth and writes the decision — "Pod X goes to Node 3" — back to the API Server.
- ♦ The Scheduler never moves cargo itself. It only assigns berths.

The Controller Manager — the reconcile crew

- ♦ A fleet of background workers, each watching one type of resource.
- ♦ Their job: compare **desired state** (what's in etcd) with **actual state** (what's running).
- ♦ If they differ, act. If they match, do nothing.
- ♦ The Deployment controller, the ReplicaSet controller, the Node controller — all live here.

This loop never stops. Desired $\neq$ actual $\rightarrow$ fix it. That is the whole job.

Worker Nodes — the ships

```
+-----+  
|           |  
|  WORKER  |  
|  NODE    |  
|           |  
|  kubelet  (the boatswain) |  
|  kube-proxy (signal flags) |  
|  Container Runtime (Docker) |  
|           |  
|  [ Pod ]  [ Pod ]  [ Pod ] |  
|           |  
+-----+
```

- ◆ Each worker node is a machine (physical or virtual) that runs Pods.
- ◆ The **kubelet** is the boatswain on each ship: it receives orders from the control plane and carries them out.
- ◆ Every node reports its health back to the API Server continuously.

The kubelet — the boatswain on deck

- ♦ The kubelet runs on every worker node. It does not run on the control plane.
- ♦ Its job: watch for Pods assigned to its node, start them, and keep them running.
- ♦ If a container crashes, the kubelet restarts it — immediately, without being asked.
- ♦ It reports the Pod's status back to the API Server so etcd stays current.

The boatswain doesn't question orders. A Pod is assigned — it runs it. A container dies — it restarts it.

The reconcile loop — tied back to Day 1

You run: `kubectl apply -f pod.yaml`

|

v

API Server receives it

|

v

etcd stores DESIRED STATE

|

v

Scheduler assigns a Node

|

v

kubelet on that Node starts the Pod

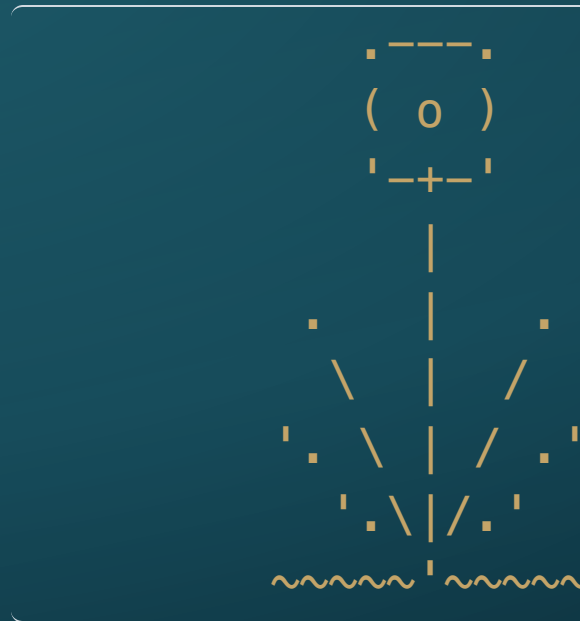
|

v

Controllers watch: `actual == desired? -> yes -> rest`

PART II

From Pod to Deployment



"One crate afloat is a start. A standing order that keeps three afloat is a fleet." — the Boatswain

The bare Pod's fatal flaw

- ♦ On Day 1 you ran a bare Pod — one manifest, one container, one copy.
- ♦ It has no safety net: if the node it lives on goes down, **the Pod stays dead**.
- ♦ No one restarts it. No one moves it. The controller manager has nothing to watch.
- ♦ A bare Pod is fine for learning. It is not how you run anything that matters.

A lone ship with no relief crew is not a fleet — it's a gamble.

A Deployment: the standing order

- ♦ A **Deployment** is a standing order to the cluster:
 - *"Always keep N copies of this Pod afloat."*
- ♦ You describe the Pod template and the replica count. The Deployment controller does the rest.
- ♦ The controller manager's Deployment controller watches it continuously.
- ♦ Lose a Pod — the controller notices within seconds and schedules a replacement.

You stop managing individual Pods. You manage the order instead.

The ReplicaSet underneath

```
Deployment
|
+-- ReplicaSet (current version of the template)
    |
    +-- Pod (replica 1)
    +-- Pod (replica 2)
    +-- Pod (replica 3)
```

- ♦ A Deployment manages a **ReplicaSet**. A ReplicaSet manages a set of identical Pods.
- ♦ You interact with the Deployment. The ReplicaSet is the machinery underneath.
- ♦ You rarely need to touch a ReplicaSet directly — the Deployment does it for you.

Self-healing in practice

- ♦ Delete one of the three Pods manually — watch what happens in k9s.
- ♦ The ReplicaSet controller notices: *actual* = 2, *desired* = 3.
- ♦ Within seconds, a new Pod is scheduled and started.
- ♦ You didn't do anything. The reconcile loop did.

```
kubectl delete pod my-fleet-abc123    <- you
delete one
# 5 seconds later...
kubectl get pods                        <- three
are running again
```

Scaling: change the order

- ◆ Need more ships? Change one number.

```
kubectl scale deployment my-fleet --replicas=5
```

- ◆ The Deployment controller sees: *desired* = 5, *actual* = 3. It schedules two more Pods.
- ◆ Scale back down: two Pods are gracefully terminated.
- ◆ No downtime. No rebuild. No manual work.

The harbour-master doesn't ask why. It just keeps the count right.

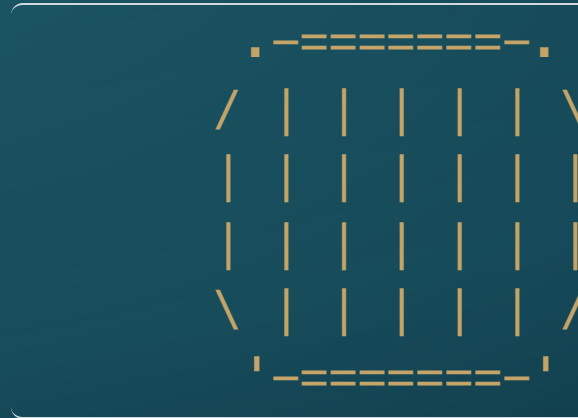
Rolling updates — no downtime

- ♦ Update your image tag in the Deployment manifest and apply it.
- ♦ Kubernetes replaces Pods **one at a time**, not all at once:
 - start one new Pod → wait for it to be healthy → terminate one old Pod → repeat
- ♦ Traffic keeps flowing through the live Pods while the swap happens.
- ♦ Roll it back just as easily if the new version misbehaves.

Like rotating a ship's crew at port — the ship never stops sailing.

PART III

Config Lives Off the Ship



"A sailor who paints his orders on the hull can't change course without a chisel." — the Boatswain

The problem with baked-in config

- ♦ The obvious place to put config is **inside the container image**.
- ♦ Hard-code `SHIP_SPEED=12` in the Dockerfile, build the image, done.
- ♦ Until the value changes. Now you:
 - edit the file
 - rebuild the image
 - push to Harbor
 - update the Deployment
 - wait for a rolling restart
- ♦ One config line change. Full rebuild pipeline.

The fix: a ConfigMap

- ♦ A **ConfigMap** stores configuration **in the cluster** — outside the image.
- ♦ It is just a Kubernetes object: key-value pairs you apply with `kubectl apply`.
- ♦ Your Deployment references it; the kubelet injects the values when it starts the Pod.
- ♦ Change the ConfigMap and roll the Deployment — **no image rebuild**.

ConfigMap		Deployment
MESSAGE_OF_THE_DAY	-->	injected as env var --> Pod sees it
SHIP_SPEED	-->	injected as env var --> Pod sees it

What a ConfigMap looks like

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: fleet-config
data:
  MESSAGE_OF_THE_DAY: "Beware the Kraken!"
  SHIP_SPEED: "12"
```

- ♦ `kind: ConfigMap` — Kubernetes knows what this is.
- ♦ `data:` — plain key-value pairs. No encryption, no secrets. Just config.
- ♦ Apply it like any other manifest: `kubectl apply -f configmap.yaml`

Injecting it into a Deployment

```
containers:
- name: web
  image: nginx:alpine
  env:
  - name: MESSAGE_OF_THE_DAY
    valueFrom:
      configMapKeyRef:
        name: fleet-config
        key: MESSAGE_OF_THE_DAY
```

- ♦ Each env var you want gets one `valueFrom` block.
- ♦ The kubelet reads the ConfigMap and injects the value when it starts the container.
- ♦ **Same image. Different ConfigMap. Different behaviour per environment.**

Why this matters for educators

- ♦ Containerize an assignment: the image is the problem set.
- ♦ The ConfigMap is the per-student variation: different parameters, different data, different challenge.
- ♦ Same image, thirty ConfigMaps, thirty distinct experiences — **no per-student rebuild.**
- ♦ The IT department isn't in this loop at all.

The lesson is a standing order. The parameters live off the ship.

PART IV

The Radar Room



kubectl works — but it's verbose

- ♦ `kubectl` is powerful and precise. It is also one command per question.
- ♦ Want to watch Pod status live? `kubectl get pods --watch`
- ♦ Want to see logs? `kubectl logs <pod-name>`
- ♦ Want to exec in? `kubectl exec -it <pod-name> -- sh`
- ♦ Each is its own command. Each requires knowing the exact Pod name. Each produces raw text.
- ♦ When you're debugging under pressure, this gets tedious fast.

Meet k9s — the terminal dashboard

```
+-----+
|  k9s   | Cluster: uss-nitic   | NS: your-name |
+-----+
| NAME           READY  STATUS   RESTARTS  AGE |
| my-fleet-abc12  1/1    Running   0          2m  |
| my-fleet-def34  1/1    Running   0          2m  |
|> my-fleet-ghi56 0/1    Error     3          45s |
+-----+
| l=logs  s=shell d=describe ctrl-d=delete |
+-----+
```

- ♦ **k9s** is a terminal UI (TUI) — a live, interactive dashboard for your whole cluster.
- ♦ One screen. Every Pod. No typing of names.

Navigating k9s

- ♦ Launch it: type `k9s` and press Enter.
- ♦ `:ns` — switch to the namespace list; arrow-key to yours and Enter.
- ♦ `/` — filter: type any string to narrow the view instantly.
- ♦ Arrow keys to highlight a Pod, then:
 - `l` — view **logs** (live-streaming)
 - `s` — **shell** into the Pod (exec)
 - `d` — **describe** the Pod (events, conditions)
 - `Ctrl-d` — **delete** the Pod
- ♦ `Esc` to back up one level. `q` to quit.

Reading the radar: Pod status at a glance

Status	What it means
Running	All containers alive and healthy
Pending	Scheduler hasn't placed it yet
CrashLoopBackOff	Container keeps crashing; K8s keeps restarting
Error	Container exited with a non-zero code
Terminating	Being gracefully shut down

- ♦ k9s colour-codes these. `CrashLoopBackOff` stands out immediately.
- ♦ When you see it: press `l` and read the logs — the answer is almost always there.

The log view

- ♦ Highlight a Pod, press `l` — logs stream live from the container.
- ♦ `s` key toggles auto-scroll on and off (useful when the output is fast).
- ♦ Arrow keys scroll up through older output.
- ♦ `Esc` returns you to the Pod list.

The error message is almost always in the last few lines of the log.

```
leaky-ship: starting up...  
leaky-ship: ERROR: cat: can't open '/nonexistent/config.txt'  
leaky-ship: exiting.
```

Exec — boarding the ship

- ♦ Highlight a Pod, press `s` — an interactive shell opens **inside** the container.
- ♦ Useful for:
 - checking that env vars injected from a ConfigMap actually arrived
 - poking at the filesystem
 - running quick diagnostics without leaving the cluster
- ♦ `exit` to leave the shell and return to k9s.

```
# inside the pod
env | grep MESSAGE
MESSAGE_OF_THE_DAY=Beware the Kraken!
```

Instructor Superpower

THE TEACHER'S DASHBOARD

- ◆ Once students know k9s, the instructor gains something more valuable: **full visibility from the podium.**
- ◆ Type `/` then a student's namespace name — their Pods, status, and logs appear instantly.
- ◆ No walking the room. No leaning over shoulders. No "can you share your screen?"
- ◆ A student's Pod in `CrashLoopBackOff`? You see it before they raise their hand. Spot the same error on three screens — address it once, for the room.
- ◆ This kind of visibility used to be a request to IT. Now it's one terminal on a cluster you already run.

Up next: the labs

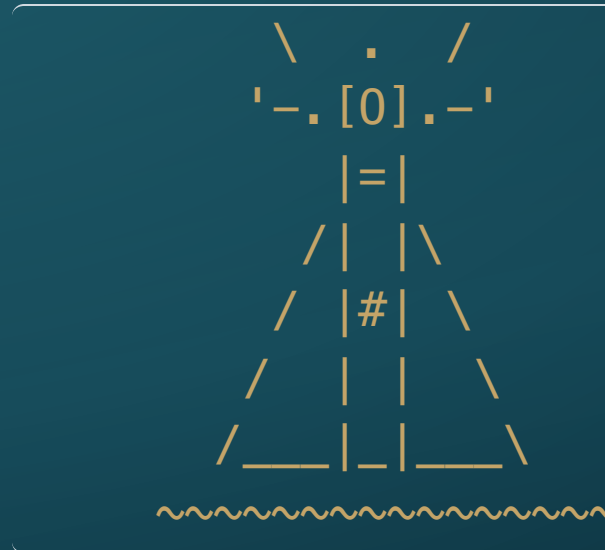
Lab 01 — The Radar Room *(right after this)*

- ♦ Open k9s. Navigate to your namespace.
- ♦ A rogue wave has hit the fleet — a broken Pod is waiting in your namespace.
- ♦ Find it, read the logs, identify the exact error.
- ♦ Then try to delete it with `Ctrl-d`. See what happens.

Lab 02 — Deploying the Fleet *(after the break)*

- ♦ Generate a Deployment manifest with `kubectl create --dry-run`.
- ♦ Scale it to 3 replicas. Watch self-healing happen live.
- ♦ Create a ConfigMap with `MESSAGE_OF_THE_DAY`, inject it into the Deployment.
- ♦ Verify inside the running Pod with `env | grep MESSAGE`.

To the radar room.



The harbour is alive. Open k9s and read the water.

